SOFTWARE REQUIREMENTS ENGINEERING (SRE)

COMPLETE COURSE MASTER HANDBOOK (HANDOUTS #1 TO #7 + INTEGRATED CLASS NOTES)

Instructor: Sir Hamood Ur Rehman Durrani | Assistant Prof. of Software Engineering | IMSciences, Peshawar

Executive Overview & Sir Hamood's Midterm Exam Blueprint

This master reference document synthesizes the entire Software Requirements Engineering (SRE) course curriculum from Handout #1 through Handout #7. It seamlessly integrates official lecture slide specifications with verbatim classroom explanations, board notes, practical real-world examples, and exam hints recorded in student class notes.

> **SIR HAMOOD'S MIDTERM PAPER PATTERN (30 MARKS TOTAL)**
>
> According to official class guidance, the Midterm Exam is divided into three equal cognitive levels (10 marks each):
>
> 1. **1/3 Direct Recall / Memory-Based (10 Marks):** Fundamental definitions, processes, and categories (e.g., RE activities, requirement types, CMM levels).
> 2. **1/3 Analytical Questions (10 Marks):** Identifying hidden errors, missing conditions, or requirement amalgamation (e.g., missing quadratic formula conditions, vague qualitative terms).
> 3. **1/3 Evaluative / Scenario-Based (10 Marks):** Given a real-world scenario, propose the appropriate architecture, prototyping strategy, or RE process (e.g., selecting horizontal vs. vertical prototyping, resolving stakeholder conflicts).

Course Map & Table of Contents

Chapter	Topic Coverage	Location
Chapter 1	Handout #1: Introduction to Requirements & Functional Requirements	Page 1
Chapter 2	Handout #2: Non-Functional Requirements (NFRs) & McCall's Quality Model	Page 2
Chapter 3	Handout #3: Domain, Inverse, Constraints & Quantitative NFR Metrics	Page 3
Chapter 4	Handout #4: User vs. System Contract Requirements, Problems & Use Cases	Page 4
Chapter 5	Handout #5: Requirements Engineering Process & Variability Factors	Page 5
Chapter 6	Handout #6: RE Elicitation, Analysis Perspectives & Social/Cultural Issues	Page 6
Chapter 7	Handout #7: Requirements Management, Validation, Formal Methods & CMM	Page 7

1.1 Definition & Purpose of Requirements Engineering

Requirements form the absolute basis for all software engineering products. **Software Requirements Engineering (SRE)** is the systematic process of identifying, analyzing, documenting, validating, and managing the services and constraints of a software system. As emphasized in class, SRE is not merely about inputs and outputs—it is fundamentally about determining whether a capability is *useful and genuinely needed* by the business.

1.2 Needs vs. Wants & The Fred Brooks Principle

A critical distinction in SRE is separating user **wants** (surface-level desires) from business **needs** (core operational necessities). Requirements engineers must always focus on satisfying true needs.

Fred Brooks' Principle: *'The hardest single part of building a software system is deciding precisely what to build... No other part of the work so cripples the resulting system if done wrong. No other part is more difficult to rectify later.'* Requirement errors caught after deployment can cost up to 100x more to fix than those resolved during early elicitation.

1.3 WHAT vs. HOW (Levels of Detail)

Requirements specify **WHAT** the system should do from the user's external perspective (external behavior), leaving the implementation code (**HOW**) hidden. In class, Sir Hamood noted: *'One person's floor is another person's ceiling'*—what serves as a detailed requirement for a manager (floor) might be considered high-level scope by a software developer (ceiling).

1.4 Functional Requirements (FRs)

Functional Requirements are statements describing the specific services, functions, and behavioral reactions of the software system to particular inputs under defined situations. They capture:

• **Reaction to Particular Inputs:** What output is generated given a specific input.

• **Behavior in Particular Situations:** How the system responds during high load or operational state shifts.

• **Sequencing & Parallelism:** Dependencies between operations (sequential execution) and independent concurrent tasks (parallel processing). In class notes, coupling is linked to sequencing while cohesion is maximized via parallel modules.

• **Exception Handling:** Documenting abnormal system behavior, user mistakes, or system failures.

1.5 Completeness, Ambiguity & Exam Case Studies

Functional requirements must be strictly **complete** (covering all legal input scenarios) and **consistent** (containing zero logical contradictions).

CLASSROOM CASE STUDIES & ANALYTICAL EXAM QUESTIONS

1. The Quadratic Formula Error (Slide 23): The requirement states: *x = (-b ± √(b² - 4ac)) / 2a*.
The Flaw: It omits the edge case when *a = 0* (which results in division by zero). A complete requirement must state: 'If a = 0, the system shall notify the user and solve as a linear equation x = -c/b.'

2. Vague Viewer Terminology (Slide 25): *'The system shall provide appropriate viewers for users to read documents.'*
The Flaw: The term 'appropriate viewers' is ambiguous. It must explicitly list supported file extensions (.pdf, .docx, .png).

3. Patient Database Search (Slide 24): Demonstrates subset filtering (admitted vs. asthma patients) requiring clear boolean search logic.

Chapter 2: Handout #2 — Non-Functional Requirements (NFRs) & McCall's Model

2.1 System-Wide Scope & Emergent Properties

While functional requirements describe individual features, **Non-Functional Requirements (NFRs)** define quality attributes, timing constraints, and operational boundaries for the **system as a whole**. They capture *emergent properties*—behaviors that manifest only when all system components run together.

Critical Nature: NFRs are often far more critical than individual functional features. Failing an NFR can render an entire software system completely unusable, even if all functional features were coded perfectly.

2.2 The 3 Major Categories of NFRs (Midterm Exam Question #4 Focus)

Sir Hamood's lectures and midterm papers organize NFRs into 3 primary categories:

Category	Sub-Types	Class Definitions & Examples
Category	Sub-Types	Class Definitions & Real-World Examples
1. Product Requirements	• Usability • Efficiency (Performance & Space) • Reliability • Portability	**Usability:** Defined by 3 things: Easy to understand, easy to learn, easy to operate. **Efficiency:** Minimize RAM memory footprint for data and instructions. Performance = speed/throughput (100k hits/min); Space = RAM consumption. **Reliability:** Output dependably satisfies business goals. Downtime < 1s per 1,000 hrs execution. **Portability:** Operates across hardware/OS environments without losing efficiency.
2. Organizational Requirements	• Standards • Implementation • Delivery	**Standards:** Process compliance (e.g., CMM, CMMI, MIL-STD-2167A). **Implementation:** Operational cutover strategy: (1) *Parallel Run* (old & new run together, outputs compared); (2) *Phase Cutover* (module-by-module transition); (3) *Direct Replacement* (instant switch). **Delivery:** Online download, physical installation, or embedded hardware deployment.

Category	Sub-Types	Class Definitions & Examples
3. External Requirements	• Interoperability • Ethical • Legislative (Privacy & Safety)	**Interoperability:** Ability to connect/integrate with third-party external systems. **Ethical:** Respecting moral standards (e.g., avoiding biased algorithms). **Legislative:** Compliance with government safety rules and data privacy laws (e.g., protecting library member data).

2.3 McCall's Quality Factors Model (Slide 16 / Assignment 1)

McCall's Quality Model (11 Factors) bridges the gap between vague quality talk and concrete software engineering metrics. It categorizes software quality into 3 perspectives (visualized as a tree with Canopy = Quality Software, Trunk = Perspectives, Roots = Factors):

Perspective	Quality Factors	Core Focus
Perspective	Quality Factors	Core Question / Focus
1. Product Operation (Day-to-day execution)	• Correctness • Reliability • Efficiency • Integrity • Usability	How well does the software run day-to-day? Does it do what it should? Is it secure (0 unauthorized leaks)? Is it efficient and user-friendly?
2. Product Revision (Modification & maintenance)	• Maintainability • Flexibility • Testability	How easily can developers fix bugs (maintainability), add new features (flexibility), and test software (testability)?
3. Product Transition (Adaptability to new env.)	• Portability • Reusability • Interoperability	How easily can software move to new platforms (portability), reuse components (reusability), and talk to other software (interoperability)?

MCCALL'S MATHEMATICAL QUALITY FORMULA (CLASS NOTES)

In class notes, McCall's model evaluates a quality factor using the formula:

FQ = c1 · m1 + c2 · m2 + c3 · m3 + ... + cn · mn

Where **FQ** = Factor Quality score, **mi** = normalized quantitative metric (0 to 1), and **ci** = weighting coefficients reflecting business priority (where ∑ ci = 1).

Chapter 3: Handout #3 — Domain, Inverse, Constraints & Quantitative NFR Metrics

3.1 Quantitative NFR Metrics Table (Slides 6–12)

Vague goals (e.g., *'the system should be fast'*) cause misinterpretation and testing disputes. To make NFRs verifiable, they must be attached to specific quantitative metrics:

Property	Verifiable Metrics	Example Verifiable Requirement
System Property	Verifiable Metrics / Measures	Example Verifiable NFR Requirement

Property	Verifiable Metrics	Example Verifiable Requirement
Speed / Performance	1. Transactions/second 2. Response time 3. Screen refresh time	Search queries shall return results within 2 seconds for 95% of requests.
Size	1. KBytes (RAM/disk) 2. Function points / LOC	Application memory footprint shall not exceed 100 MB during execution.
Ease of Use	1. Training time (hours) 2. Number of help frames	New operators shall achieve competency after 2 hours training with < 2 errors/day.
Reliability	1. Mean Time To Failure (MTTF) 2. Availability % 3. Rate of failure	System uptime shall be 99.9% with MTTF exceeding 1,000 continuous hours.
Robustness	1. Time to restart after failure 2. % events causing failure 3. Data corruption probability	System shall restart within 30 seconds following power failure without data corruption.
Portability	1. % target-dependent lines 2. Number of target OS platforms	At least 95% of source code shall be OS-independent across Windows and Linux.

3.2 High Cost of Quantitative NFR Verification (Slide 13)

Class notes emphasize that while quantitative metrics make testing objective, **the cost of quantitatively verifying each NFR can be extremely high** due to expensive stress-testing servers, long test durations (e.g., 1,000 hours continuous execution), and human user trials.

3.3 Domain Requirements

Domain Requirements originate directly from the application domain (e.g., medical, railway, aviation, banking) and reflect fundamental characteristics and legal/physical rules of that domain. They can be functional or non-functional.

• **Implicit Nature:** Domain experts take these rules for granted and find it difficult to convey them explicitly. Omitting them causes severe user dissatisfaction.

• **Medical Domain Example (Slide 18):** Insulin pump safety compliant with standard *IEC 60601-1*.

• **Railway Domain Example (Slide 19):** Train deceleration formula: *D(train) = D(control) + D(gradient)*, where *D(gradient) = 9.81 m/s² * compensated gradient / α*.

• **Sales Domain Example (Slide 21):** Non-negative commission rule (commission can never be negative).

• **IMSciences University Domain Example:** Inter-departmental business flow: Admissions Registration Exams Finance Clearance Degree Awarding.

3.4 Inverse Requirements & Implementation Constraints

• **Inverse Requirements:** Explicitly specify *what the system shall NOT do* (e.g., 'The system shall not use red color in input screens', 'The system shall not allow negative cash balances'). They reflect cautious or indecisive customer preferences.
• **Design & Implementation Constraints:** Technical guidelines restricting development choices (e.g., mandating .NET framework, open-source Linux stack, or specific database tools). They directly impact developer skill hiring requirements.

4.1 User Requirements vs. System Contract Requirements

Handout #4 introduces an alternative categorization based on target audience and technical detail:

View	Audience	Characteristics	Purpose
Requirement View	Target Audience	Format & Characteristics	Core Purpose
User / Customer Requirements	End-users, clients, non-technical managers	Natural language, simple diagrams, forms. Avoids technical system design jargon.	Describes expected services and business constraints from customer perspective.
System Contract Requirements	Software engineers, developers, technical architects	Detailed, technical natural language, UML models, mathematical specs.	Sets out detailed services and constraints. Forms the legal contract and architectural baseline.

4.2 Traceability in 3 Directions (Class Exam Hotspot)

To manage requirement changes effectively, every system contract requirement must maintain **Requirements Traceability** across 3 distinct dimensions:

1. **Requirement-to-Source Traceability:** Links the requirement back to the specific stakeholder, business process, or regulatory policy that requested it.

2. **Requirement-to-Requirement Traceability:** Links dependent requirements to each other (e.g., showing which sub-features depend on a core authentication module).

3. **Requirement-to-Design Traceability:** Links the requirement directly to the specific software architecture module or design class (ensuring developers know exactly which code files to modify when a requirement changes).

4.3 Problems with Natural Language Requirements

Relying solely on natural language causes severe RE problems: (1) *Lack of Clarity*; (2) *Requirement Confusion* (mixing FRs, NFRs, and design details); (3) *Requirement Amalgamation* (combining multiple unrelated constraints into one long sentence, making modular design impossible); (4) *Overflexibility* (vague wording open to conflicting interpretations).

4.4 Craig Larman's Use Case Detail Levels (Class Notes)

Derived from Craig Larman's *Applying UML and Patterns*, class notes specify 3 levels of Use Case detail:

• **Brief Format:** A single-paragraph summary of the main success scenario (written during early scoping).

• **Casual Format:** Informal multi-paragraph text covering main and alternate scenarios.

• **Fully Dressed Format:** A rigorous structured template containing: Use Case Name, Primary Actor, Stakeholders & Interests, Preconditions, Postconditions / Success Guarantees, Main Success Flow (Normal Flow), Alternate Flows / Extensions, and Business Rationale.

5.1 RE Process Inputs & Outputs

The RE Process is a continuous, non-sequential set of activities that transforms raw stakeholder wishes into formal system specifications.

RE Process Inputs	RE Process Outputs
RE Process Inputs	RE Process Outputs
• Existing systems information (Excel, legacy DB) • Stakeholder needs & business goals • Organizational standards (CMM/CMMI) • External regulations & legislation • Domain information & business rules	• Agreed User Requirements Document • Detailed System Specification (SRS) • System Models (Use Case, Data Flow, DFD) • Requirements Traceability Matrix

5.2 The 5 High-Level RE Activities

1. **Requirements Elicitation:** Discovering requirements through stakeholder consultation, domain analysis, and document review.

2. **Requirements Analysis & Negotiation:** Categorizing requirements, identifying conflicts between stakeholders, and negotiating trade-offs based on business goals.

3. **Requirements Specification:** Documenting requirements in a formal SRS using natural language and visual models.

4. **Requirements Validation:** Reviewing the SRS for completeness, consistency, and realism before signing contracts.

5. **Requirements Management:** Tracking changes, maintaining traceability, and controlling baselines throughout software lifecycle.

5.3 RE Process Variability Factors (Past Paper Question #5)

RE processes vary radically from one organization to another based on 4 key factors:

• **Technical Maturity (CMM/CMMI Level):** Higher maturity organizations follow rigorous, standardized RE steps, whereas low maturity teams use ad-hoc methods.

• **Organizational Culture:** Corporate practices and communication habits.

• **Disciplinary Involvement:** The mix of software engineers, legal counsel, domain experts, and business analysts involved.

• **Application Domain:** A hospital safety system requires strict compliance processes, whereas a university portal emphasizes rapid UI prototyping.

6.1 Requirements Elicitation Techniques & Prototyping

Elicitation techniques include Interviews, Scenarios, Workshops, Ethnomethodology (observing users in their real environment), Mind Mapping, and Prototyping.

• **Horizontal Prototyping:** Covers the breadth of the system interface (GUI mockup) across all modules without real database/backend logic.

• **Vertical Prototyping:** Implements a complete slice of functionality through all architectural layers (GUI + Backend + DB) for one core feature.

• **Throwaway vs. Evolutionary Prototyping:** Throwaway prototypes explore risky requirements and are discarded; evolutionary prototypes gradually evolve into the final product.

6.2 Three Analysis Perspectives

1. **Business-Flow Perspective:** Analyzes the sequential order of business events (workflow charts).

2. **Data-Flow Perspective:** Analyzes data movement, inputs, and outputs across processes (Data Flow Diagrams - DFDs).

3. **Role-Action Perspective:** Identifies who is involved (roles) and what specific actions they perform across business processes.

6.3 Role-Action Diagram for Prototyping (Midterm Past Paper Question #8)

> **ROLE-ACTION DIAGRAM FOR SOFTWARE PROTOTYPING (PAST PAPER Q8)**
>
> **Action Sequence:** Requirement Elicitation → Negotiation & Analysis → Selecting Software Prototype → Developing Prototype → Evaluation (with feedback loops).
>
> **Role Definitions:**
>
> • *Domain Expert:* Possesses complete subject matter knowledge of business operations.
>
> • *Requirements Engineer (RE):* Gathers, structures, and analyzes requirements.
>
> • *Business Analyst:* Represents organizational business processes and objectives.
>
> • *Software Engineer:* Develops prototype model and backend code.
>
> • *Project Manager:* Manages resources, schedules, and evaluates prototype progress.
>
> • *End User:* Operates the prototype and provides direct feedback.

6.4 Social & Cultural Issues in RE (Class Notes Focus)

RE is fundamentally a **social process**. Sir Hamood's lectures highlight 6 critical areas of social/cultural friction:

1. **Social Issues within Client Org:** End-users determine ultimate system success, but middle managers may feel threatened by automation.

2. **Issues within Requirement Team:** Organization of work, role rotation, and internal communication.

3. **Client vs. RE Team Friction:** Financial delays, denial of confidential information, and scope changes.

4. **Cultural & Language Differences:** Time zone gaps in global teams and terminology misunderstandings (e.g., 'Hockey' in US = ice hockey vs. Pakistan = field hockey).

5. **Religious, Racial & Political Nuances:** Insensitive comments destroy collaboration. Local workplace norms (e.g., Friday Jumma Namaz schedules, Ramadan working hours) must be respected.

6. **The 5th Influence (GenZ / New Pakistanis):** Adapting RE communication styles to modern digital work culture.

Chapter 7: Handout #7 — Requirements Management, Validation & Process Improvement

7.1 Requirements Validation Techniques

Requirements Validation ensures the SRS accurately reflects customer needs before coding begins. Techniques include:

• **Ad-Hoc Reviews:** Informal peer readings.

• **Checklist Reviews:** Systematically scanning the SRS against standard quality checklists.

• **Defect-Based Reading:** Reviewers search for specific defect categories (omissions, ambiguities).

• **Perspective-Based Reading (PBR):** Reviewers inspect the SRS from specific stakeholder perspectives (Tester, Developer, User).

• **Formal Mathematical Specifications:** Using formal mathematical languages like *Z-Specification Language* or *VDM (Vienna Development Model)* to prove correctness without natural language ambiguity.

7.2 Requirements Management & Traceability Matrix

Requirements Management spans the entire software lifecycle. Key practices include version control, baseline tracking, and maintaining a *Traceability Matrix* to evaluate the ripple effect of proposed scope changes.

7.3 CMM / CMMI 5 Maturity Levels in RE (Past Paper Question #7)

Capability Maturity Model (CMM/CMMI) resolves RE process chaotic issues by introducing 5 technical maturity levels:

CMM Level	Process Characteristic	RE Implementation
CMM Level	Process Characteristic	RE Process Implementation
Level 1: Initial	Chaotic, ad-hoc, unpredictable	No formal RE process; reactive fire-fighting.
Level 2: Managed	Repeatable project management	Basic requirements tracking and change control.
Level 3: Defined	Organizational standardization	Formal RE process documented and followed across firm.
Level 4: Quantitatively Managed	Measured and controlled	RE processes measured quantitatively using defect metrics.
Level 5: Optimizing	Focus on continuous improvement	Proactive RE process optimization and technology innovation.

FINAL EXAM REVISION SUMMARY & STUDY ADVICE

1. **Master the Taxonomies:** Know the 5 Kinds of Requirements (FR, NFR, Domain, Inverse, Constraints) and the 3 NFR Categories (Product, Organizational, External).

2. **Practice Conversions:** Be ready to convert vague qualitative goals (e.g., 'fast user response') into verifiable quantitative NFRs with metrics.

3. **Memorize Traceability:** Be prepared to explain Requirement-to-Source, Requirement-to-Requirement, and Requirement-to-Design traceability with examples.

4. **Review Scenario Diagrams:** Be ready to draw and explain the Role-Action Diagram for Software Prototyping (Past Paper Question #8).